AIKDAP

AI Knowledge Discovery and Analytics Platform:
A Multi-Agent, Project-Scoped AI Work Operating System
with Backend-Enforced Grounding

A Thesis Submitted in Partial Fulfilment of the Requirements
for the Degree of _____

Submitted by

_____

Under the Guidance of

_____

Department of _____
_____ (Institution)
October 2026

Abstract

Knowledge workers split one task across many disconnected tools: a chat model for reasoning, a search engine for discovery, a notebook tool for reading documents, a notebook environment for data analysis and a BI tool for presentation. Each switch loses context, nothing is remembered, and the outputs are hard to verify. Retrieval-augmented generation (RAG) assistants make this worse by *asking* the model to stay grounded through a prompt instruction, which is a request, not a guarantee.

This thesis presents **AIKDAP**, an AI Work Operating System. It accepts a single task, plans it, distributes it to specialised agents orchestrated by LangGraph, validates the result and delivers a finished, traceable artifact inside a persistent project workspace. The central technical contribution is **verifiable non-hallucination**. Three mechanisms enforce grounding in backend control flow rather than in prompts: (i) a three-stage retrieval pipeline (BGE-M3 dense retrieval, a BGE-Reranker-v2-m3 cross-encoder and an empirically calibrated relevance gate); (ii) a citation validator that rejects any citation identifier not placed in the model's prompt; and (iii) a backend-computed grounding status that the model cannot set for itself. The same principle carries over to business analytics: the language model only plans and explains, deterministic pandas code computes every number, and every number in the narrative is checked against the result table.

On a fully local calibration fixture of 70 labelled (query, chunk) pairs, a gate threshold of −2.0 gives a precision of 0.818 and a recall of 0.818. Four of five out-of-corpus queries are reduced to zero evidence, so no generation call is made for them. The platform contains about 31,000 lines of backend code, about 27,000 lines of frontend code and more than 1,100 automated backend tests. It also implements three persona workflows (Student, Researcher, Project Builder) on a shared grounded substrate.

Keywords: retrieval-augmented generation, hallucination, multi-agent systems, LangGraph, cross-encoder reranking, explainable AI, provenance, business analytics.

Table of Contents

1. Introduction
2. Literature Review
3. Problem Definition and Requirements
4. System Architecture and Design
5. Implementation
6. Testing and Evaluation
7. Discussion and Limitations
8. Conclusion and Future Work
9. References

Chapter 1 — Introduction

1.1 Background

Large language models (LLMs) have changed how people search, read and write. However, the workflow of a student, researcher or analyst is still scattered. A typical research task means finding papers in a search engine, reading them in a notebook tool, asking a chat model to summarise them, moving datasets into a notebook to analyse, and finally assembling charts and slides in a BI or office tool. Each tool keeps its own context, so the user repeatedly re-explains the task, and no single tool can say how a final claim was produced.

1.2 Motivation

Two observations motivate this work. First, existing assistants are built to *answer questions*, while users need to *finish work*: a report, an analysis, a synopsis, a build plan. Second, the answers these assistants give cannot be verified structurally. A RAG system that tells the model to "use only the provided context" can still cite a document it never saw, and a top-k retriever asked for five chunks always returns five, however irrelevant they are. In our deployment, the question "What is the capital of Japan?" retrieved five poultry-industry chunks at a cosine similarity of about 0.24. A system that feeds such evidence to a generator invites fabrication.

1.3 Aim

To design and build a single project-scoped workspace in which a user gives one task and a multi-agent system plans, executes, validates and delivers the complete result, with every claim traceable to its evidence and every step recorded.

1.4 Objectives

1. Replace the fragmented tool chain with one workspace organised around *projects*.
2. Break tasks into steps automatically through intent analysis and a planner agent.
3. Orchestrate specialised agents with LangGraph, with all communication through a typed shared state.
4. Enforce grounding in backend code: calibrated relevance filtering, citation validation and a backend-computed grounding status.
5. Provide deterministic, verifiable business analytics on user datasets.
6. Keep memory per project and treat every output as a reusable asset.
7. Record a complete, queryable execution trace for every workflow (explainable AI).

8. Serve three user personas (Student, Researcher, Project Builder) from one shared substrate.

1.5 Scope

The scope covers the backend platform (authentication, projects, assets, knowledge base, research, analytics, reports, workflows, notifications), the agent graphs, a React frontend and Docker-based deployment. Training new models is out of scope; the system composes existing open models (Qwen 3.5 4B, BGE-M3, BGE-Reranker-v2-m3) and OpenAI-compatible cloud LLMs.

1.6 Contributions

- A **backend-enforced grounding chain**: over-retrieve, rerank, then apply a calibrated gate, with citation validation that drops unknown identifiers instead of repairing them.
- An **empirical calibration method** for an unbounded cross-encoder logit threshold, together with an argument for rejecting the $F_{0.5}$-optimal threshold.
- **Verifiable analytics**: the LLM plans and explains, deterministic code computes, and every narrated number is checked against the result table.
- A **resilient LLM gateway** that classifies quota windows rather than trusting provider retry hints, with a circuit breaker shared through Redis.
- **Structural explainability**: any run can be reconstructed node by node from persisted tables.
- A **persona layer** showing that one grounded substrate serves three different workflows.

1.7 Organisation of the Thesis

Chapter 2 reviews related work. Chapter 3 defines the problem and requirements. Chapter 4 presents the architecture, and Chapter 5 the implementation. Chapter 6 reports testing and evaluation, Chapter 7 discusses limitations, and Chapter 8 concludes.

Chapter 2 — Literature Review

2.1 Large Language Models and Hallucination

Transformer-based LLMs [1] produce fluent text but are known to hallucinate, stating content that is unsupported by or contradicts their sources [2]. Surveys separate intrinsic hallucination (contradicting the source) from extrinsic hallucination (unverifiable additions). Both matter in research settings, where a fabricated figure or citation can spread into downstream work.

2.2 Retrieval-Augmented Generation

RAG [3] conditions generation on retrieved passages and reduces, but does not remove, hallucination. Dense passage retrieval [4] made embedding-based retrieval standard. Most RAG pipelines keep grounding inside the prompt and take a fixed top-k, so they cannot say "no relevant evidence exists".

2.3 Two-Stage Retrieval and Cross-Encoder Reranking

Two-stage retrieval, a fast bi-encoder followed by a more accurate cross-encoder [5], is well established in information retrieval. BGE-M3 [6] provides multilingual, multi-granularity embeddings, and the BGE reranker family scores (query, passage) pairs jointly. Cross-encoder scores are unbounded logits, so a relevance threshold has to be calibrated for the deployment rather than assumed.

2.4 Attribution and Citation Verification

Work on attributed generation evaluates whether generated statements are supported by cited sources [7]. Most systems check attribution after the fact with another model. AIKDAP instead treats citation validity as a *set-membership* property checked deterministically against the evidence actually provided.

2.5 LLM Agents and Orchestration

ReAct [8] and later agent frameworks interleave reasoning and tool use. Multi-agent frameworks such as AutoGen [9] let agents converse directly, which makes their traces hard to audit. LangGraph [10] models a workflow as an explicit state graph. AIKDAP adopts this model and adds a strict rule: agents never call each other and communicate only through shared state.

2.6 Natural-Language Data Analysis

Text-to-SQL and "chat with your data" tools usually let a model generate executable code, which is unsafe and hard to verify. AIKDAP limits the model to emitting a schema-validated analysis plan. Each plan field maps to one explicit pandas operation, with no `eval`.

2.7 Explainable AI

Explainable AI (XAI) research [11] focuses mostly on explaining individual model predictions. For agentic systems, a more practical form of explanation is *process provenance*: what was planned, which agent ran, with what inputs, and how long it took. AIKDAP treats this as a database schema rather than a narration generated by the model.

2.8 Existing Tools and the Research Gap

	ChatGPT	**Perplexity**	**NotebookLM**	**Elicit/SciSpace**	**AIKDAP**
Grounded in user documents	partial	no	yes	yes	yes
Citation validity enforced in code	no	no	no	no	yes
Calibrated zero-evidence outcome	rarely	no	rarely	rarely	yes
Provenance to chunk + raw scores	no	link	passage	passage	yes
Persisted execution trace	no	no	no	no	yes
Verified dataset analytics	code interp.	no	no	no	yes
Unit of memory	chat	query	notebook	query	project

Table 2.1 — Comparison with existing tools (author's assessment of publicly documented behaviour)

Gap: no existing tool combines project-scoped memory, multi-agent completion of whole workflows, and grounding enforced *structurally* rather than by prompt.

Chapter 3 — Problem Definition and Requirements

3.1 Problem Statement

Design a platform that takes a single natural-language task within a project, completes the research or analytics workflow behind it, and returns a deliverable whose every claim is either traceable to stored evidence or explicitly marked as unsupported. The platform must remain usable when individual models or providers fail.

3.2 Specific Problems Addressed

1. **Unverifiable output:** prompt-level grounding cannot stop citations to evidence that was never retrieved.
2. **Top-k always returns k:** ordering is confused with relevance.
3. **Silent provider fragility:** daily quotas look like transient errors and naive retry loops never succeed.
4. **Fragmented context:** chat-scoped memory loses prior work.
5. **Untraceable process:** users cannot see how an answer was produced.

3.3 Functional Requirements

ID	Requirement
FR1	Register and authenticate users with JWT (access and refresh tokens).
FR2	Create, read, update and delete projects; all data is owned through a project.
FR3	Upload assets (PDF, DOCX, PPTX, XLSX, HTML, CSV); extract, OCR, chunk, understand and embed them.
FR4	Run grounded research over project assets with validated citations.
FR5	Suggest external papers (OpenAlex) when evidence is insufficient, and let the user add them and re-run.
FR6	Analyse tabular datasets (upload or Kaggle import) from natural-language questions, with charts.
FR7	Generate reports, synopses (DOCX/PDF) and build plans according to persona.
FR8	Persist an execution trace for every run and expose it to the user.
FR9	Provide a Command Center showing current activity across projects.

3.4 Non-Functional Requirements

- **Trustworthiness:** no stored citation may reference evidence that was not supplied to the model.
- **Resilience:** declared and reported degraded paths for each model or provider failure.

- **Security:** JWT authentication, secrets only in environment variables, ownership checks at every layer, no execution of generated code.
- **Performance:** async I/O; long-running work runs on Celery workers.
- **Maintainability:** feature-based modules (router / service / schemas / models / repository), type hints and Alembic migrations.
- **Privacy and cost:** ingestion, embedding, reranking and understanding run locally.

3.5 Users and Personas

Persona	Goal	Primary deliverables
Student	Turn a paper into coursework artifacts	Study Summary (DOCX/PDF)
Researcher	Continue a line of research	Grounded answers, sufficiency verdict, gap list, cross-paper comparison, paper suggestions
Project Builder	Implement what a paper describes	Project Synopsis, Build Plan (tools and process)

Chapter 4 — System Architecture and Design

4.1 Enterprise Workflow

```
User → Authentication → Intent Analysis Agent → Planner Agent
     → LangGraph Orchestrator → Specialized AI Agents → Tool Registry
     → Knowledge Layer → Response Aggregator → Validation Service
     → Explainable AI → Final Response
```

Figure 4.1 — End-to-end workflow

4.2 Layered Architecture

```
React + TypeScript SPA (Vite, Tailwind, shadcn/ui, TanStack Query)
   |  REST /api/v1  (JWT)
FastAPI  — feature modules: router → service → repository → model
   |-- Celery workers (Redis broker): ingestion, research, analytics, reports
   |-- LangGraph orchestrators: research, analytics, report, build-plan graphs
   |-- LLM Gateway (single LiteLLM boundary; retry, fallback, circuit breaker)
   \-- Knowledge layer: PostgreSQL 17 + pgvector, Redis, ChromaDB
```

Figure 4.2 — Layered architecture

The backend is organised into feature modules: `auth`, `users`, `projects`, `assets`, `knowledge_base`, `research`, `papers`, `business_analytics`, `reports`, `execution`, `tasks`, `workflows`, `notifications`, `health`. Every module uses the same five-file structure, and business logic never sits in routers. Ownership is enforced through the owning project. "Exists but is not yours" returns the same response as "does not exist", so resource identifiers are not leaked.

4.3 Project-Centric Information Model

Each project contains an Overview, Research Workspace, Business Analytics, Knowledge Base, Assets, Reports, Workflow Timeline and Settings. Memory is per project: Redis holds short-term state, and PostgreSQL (with pgvector) and ChromaDB hold long-term state. Every generated artifact (report, synopsis, chart, analysis, build plan) becomes a persistent asset.

4.4 Agent Orchestration Principles

- **Planner decides, LangGraph orchestrates.** The topology is explicit, and routing functions choose the next node.
- **Agents never call each other.** A node only returns a partial update to a typed shared state (a `TypedDict`). This is what makes the trace complete.

- **Reducers.** Accumulating channels (documents, messages) use `operator.add`; all other keys are last-write-wins.
- **Stateless compiled graphs.** The database session, strategies and tracker are injected per invocation through `config["configurable"]`, so one compiled graph is safely shared across concurrent runs.
- **Uniform instrumentation.** Every node is wrapped by `instrument()`, which times it, reports start, success and failure, and applies the failure policy declared in the registry.
- **Honest degradation.** When a non-critical node fails, it writes a failure marker, and synthesis must state in the deliverable that the source was unavailable.

4.5 Agent Graphs

Research graph

```
START → planner → router ─┬→ asset_retrieval ─┐
                          └→ web_research ────┴→ context_builder → synthesis
synthesis ─┬→ (insufficient, web not yet tried) → web_research → …
           ├→ (insufficient, OpenAlex configured) → paper_suggestion → END
           └→ END
```

Business analytics graph

```
START → load_dataset → plan_analysis → execute → explain → validate → END
```

Report (synopsis) graph

```
START → collect_documents → retrieve_evidence → write_sections → coverage_check → END
```

Build-plan graph

```
START → extract → research_further → recommend_tools → recommend_process → END
```

Figure 4.3 — The four LangGraph orchestrators

4.6 Database Design

The main tables are `users`, `projects`, `assets`, `knowledge_chunks`, `tasks`, `research_runs`, `research_steps`, `agent_messages`, together with tables for analytics, reports, papers, workflows and notifications. All tables use UUID primary keys and `created_at`/`updated_at` timestamps, and every schema change goes through Alembic. `grounding_status` is a first-class enum column, so "every run that could not be answered" is a plain SQL query. A run's `task_id` uses `ON DELETE SET NULL`, so archiving a task never erases its audit trail.

4.7 Security Design

Authentication uses HS256 JWTs (30-minute access tokens, 7-day refresh tokens). Secrets are read from the environment as `SecretStr`, are never logged, and are scrubbed from provider error text. A startup validation pass refuses to boot with an unsafe configuration. No module executes model-generated code.

Chapter 5 — Implementation

5.1 Technology Stack

Layer	Technology
API	Python 3.12, FastAPI, Pydantic v2
Persistence	SQLAlchemy 2 (async), Alembic, PostgreSQL 17 + pgvector, ChromaDB
Async work	Celery with a Redis broker
AI orchestration	LangGraph, LangChain, LiteLLM
Local models	Qwen 3.5 4B (understanding), BGE-M3 1024-d (embedding) via Ollama; BGE-Reranker-v2-m3 via llama.cpp
Cloud LLMs	OpenAI-compatible providers (Gemini, Groq, OpenRouter) behind a fallback chain
Data / maths	pandas, SymPy
Documents	pypdf, python-docx, python-pptx, openpyxl, BeautifulSoup/lxml, Tesseract OCR
Frontend	React, TypeScript, Vite, Tailwind CSS, shadcn/ui, TanStack Query
Deployment	Docker, Docker Compose

5.2 Document Ingestion Pipeline

The pipeline runs `extract → chunk → understand → embed` as a single Celery task.

- **Deterministic extraction:** no model ever "recovers" text.
- **OCR:** a PDF goes to Tesseract when more than half its pages are empty. OSD first corrects 90/180/270° rotation. Limits: confidence ≥ 40, 30 s per page, ≤ 50 pages. Tesseract was chosen over EasyOCR and PaddleOCR (multi-GB dependencies) and over PyMuPDF (AGPL licence).
- **Provenance-preserving chunking:** each page, sheet, slide or section is chunked on its own, so a chunk never crosses a page boundary. Boundaries snap to paragraph, then sentence, then word (size 1000, overlap 100).
- **Best-effort AI:** understanding and embedding failures are tracked in their own status fields and never invalidate extracted text or chunks.

5.3 Three-Stage Retrieval

Stage	Mechanism	Output
0	LLM query reformulation (schema-validated; preserves entities, metrics, dates and regions)	search query + facets
1	BGE-M3 + pgvector cosine search, ownership-filtered, candidate_k = 20	candidates + cosine distance
2	BGE-Reranker-v2-m3 cross-encoder, top_k = 5	unbounded relevance logit
3	Relevance gate, threshold −2.0	possibly zero evidence

Scores are never fused or normalised: `retrieval_score` and `rerank_score` are stored side by side on their native scales. The rerank score is left absent, not set to zero, when the reranker did not run. The reranker could not be served by Ollama, so it runs under `llama-server --reranking`. Because the client uses the standard rerank contract, this was a configuration change, not a code change.

5.4 Grounded Synthesis and Citation Validation

`GroundedSynthesizer` enforces three guarantees in code:

1. Only retrieved evidence is sent: the model gets no tools, no database access and no history.
2. Only supplied evidence can be cited: every returned citation ID is checked against the IDs placed in the prompt, and unknown IDs are **dropped and recorded, never repaired**.
3. The model does not grade itself: `grounding_status` is computed from the validated citation set.

An `ExtractiveSynthesizer` copies sentences verbatim from the evidence. It serves as an offline fallback and as an evaluation control. Evidence is capped at 12,000 characters, and generation runs at temperature 0.2 with a limit of 2,048 tokens.

5.5 Research Intelligence Layer

- **Document understanding:** extracts equations, variables, metrics and datasets, each traced to its location. It separates explicit facts from inferred ones, records internal conflicts, classifies research gaps (REQUIRED / HELPFUL / OPTIONAL / AMBIGUOUS) and gives a sufficiency verdict.
- **Cross-paper synthesis:** keeps a hard boundary between primary and supporting evidence, reports contradictions rather than resolving them, and forces hypotheses into modal language ("may improve").
- **Experiment Plan Engine:** SymPy-based equation parsing and constraint derivation that never executes generated code.

- **Paper suggestions:** when synthesis reports insufficient evidence, the graph queries OpenAlex. The user can add suggested papers to the project and re-run the research.

5.6 Business Analytics Module

The analytics graph applies the grounding principle to numbers:

1. **load_dataset:** loads an uploaded file or a Kaggle import; failed imports are shown to the user, not hidden.
2. **plan_analysis:** the LLM sees only the schema and emits a Pydantic-validated `AnalysisPlan` (filters, group keys, aggregations, time periods).
3. **execute:** each plan field maps to one explicit pandas call, with no `eval`, no `query` and no strings turned into code.
4. **explain:** the LLM narrates the result using only the result rows.
5. **validate:** every number in the narrative is extracted (including the k/M/B and % forms) and matched against the result table within a 1% relative tolerance. Unmatched numbers are flagged to the user.

Charts are generated from the result table. A guard prevents an empty chart from being rendered from point-list data.

5.7 Reports and Persona Features

Each user selects a persona (Student, Researcher or Project Builder), which a project can override. The report graph collects documents, retrieves evidence per section, writes sections from that evidence and finishes with a coverage check. It produces a **Study Summary** or a **Project Synopsis** exported as DOCX or PDF. The build-plan graph extracts what a paper requires, researches further, and recommends tools and an implementation process.

5.8 LLM Gateway and Resilience

Only one module imports LiteLLM, and a test enforces this. The motivating incident: Gemini's free-tier 429 response carries `retryDelay: "35s"` on a quota that resets *daily*, so a retry loop that honours the hint never succeeds. The gateway therefore classifies errors:

Error class	Behaviour	Cooldown
Per-minute rate limit	retry with jittered exponential backoff (1–8 s, ≤ 2 retries)	60 s
Day-scale quota	no retry; go straight to the fallback	3600 s
Balance exhausted	no retry; fallback	—
Terminal (bad credentials or request)	stop the whole chain	300 s

Requests pass through a fallback chain (for example Gemini A → Gemini B → Groq → OpenRouter). Provider health is stored in Redis so the API process and the workers share one breaker. Every response records `attempts`, `fallback_used` and the model that actually answered.

5.9 Explainability Implementation

`research_runs` stores the query, plan, objective, answer, citations, grounding status, reproducibility inputs, Celery task ID and timings. `research_steps` stores one row per node execution, and `agent_messages` stores the inter-agent transcript. A reconciliation worker marks runs stale after 1,200 s so a crashed worker never leaves a run pending forever. The frontend shows this trace as a workflow timeline and an evidence funnel.

5.10 Frontend

The single-page app is built around a **Command Center** that answers "What is happening?". It includes project views for assets and their processing timeline, a research workspace with a citation list and evidence drawer, document and cross-paper analysis panels, business analytics with charts, reports with DOCX/PDF export, and persona settings. Server state is managed with TanStack Query.

5.11 Implementation Scale

Metric	Value (October 2026)
Backend application code	≈ 31,300 lines of Python
Backend automated tests	≈ 1,160 test functions
REST endpoints	≈ 65 under /api/v1
Feature modules	14
LangGraph orchestrators	4 (research, analytics, report, build plan)
Frontend	≈ 27,400 lines across 185 TypeScript/TSX files

Chapter 6 — Testing and Evaluation

6.1 Testing Strategy

The backend is tested with pytest at three levels. Unit tests cover pure logic such as chunking, plan execution, number grounding and error classification. Service and route tests run against a PostgreSQL test database. *Architectural* tests guard design invariants, for example LiteLLM containment, docs exposure policy, the Docker execution policy and experiment safety. Representative suites include `test_relevance_gate`, `test_grounded_synthesis`, `test_fallback_grounding`, `test_research_citations`, `test_llm_resilience` and `test_research_run_reconciliation`. Static quality is enforced with ruff, black, isort and mypy, and the frontend has its own component tests.

6.2 Relevance Gate Calibration

Method. The real stack (BGE-M3 → pgvector → BGE-Reranker-v2-m3 under llama.cpp, with nothing mocked) scored every chunk against every query. The corpus had 7 chunks and 10 queries (5 in-corpus, 5 out-of-corpus), giving 70 pairs: 11 relevant and 59 irrelevant. Labels were assigned from document content, never from the score being measured.

Class	n	min	max	mean	median
Relevant	11	−10.759	+10.272	+2.296	+3.262
Irrelevant	59	−11.048	+2.133	−8.156	−10.072

Table 6.1 — Reranker score distribution

Threshold	TP	FN	FP	TN	Precision	Recall	FPR
−4.0	9	2	12	47	0.429	0.818	0.203
−3.0	9	2	8	51	0.529	0.818	0.136
−2.0	**9**	**2**	**2**	**57**	**0.818**	**0.818**	**0.034**
−1.0	8	3	2	57	0.800	0.727	0.034
0.0	6	5	2	57	0.750	0.545	0.034
+2.5	6	5	0	59	1.000	0.545	0.017

Table 6.2 — Threshold sweep

Result. At −2.0, precision jumps from 0.529 to 0.818 while recall stays at its maximum. All five in-corpus queries keep at least one genuinely relevant chunk with no false accepts, and **four of five out-of-corpus queries drop to zero evidence, so no synthesis call is made**. The $F_{0.5}$-optimal threshold (+2.25, precision 1.0, recall 0.545) was rejected because it

turns a question the corpus does answer into a false "insufficient evidence" result. For a research assistant, answering real questions matters more than an aggregate metric.

6.3 Citation Validation

Tests inject model outputs that cite identifiers which were never supplied. In every case the identifier is dropped and recorded, and a response that claims grounding with no valid citations is stored as `insufficient_evidence`. This property holds by construction: a stored citation is always a member of the supplied evidence set.

6.4 Analytics Number Grounding

Tests cover the formats of numbers that appear in narratives (thousands separators, decimals, percentages, k/M/B suffixes). A narrative that states a value absent from the result table is flagged, and values within 1% relative tolerance (for example "1.23M" against 1,234,567.89) are accepted.

6.5 Resilience

Tests simulate each error class and check that the gateway behaves as designed: per-minute limits are retried, day-scale quotas skip directly to the fallback without retrying, terminal errors stop the chain, and the breaker state stored in Redis is honoured across processes. The outcome is recorded on every response.

6.6 Example End-to-End Outcomes

Scenario	Observed behaviour
In-corpus research question	Grounded answer; citations resolve to chunk, page and file, with both scores
Out-of-corpus question ("capital of Japan" over a poultry corpus)	Zero evidence passes the gate → insufficient evidence → OpenAlex suggestions offered
Reranker offline	Retrieval order kept; `reranking_status` reported
Primary LLM daily quota exhausted	Fallback provider answers; `fallback_used = true` recorded
Analytics question on a sales CSV	Deterministic table and chart; narrative numbers verified
Failed Kaggle import	Failure shown to the user instead of an empty analysis

Chapter 7 — Discussion and Limitations

7.1 Discussion

The results support the thesis that grounding works better as a **property of the control flow** than as an instruction to the model. The language model is still useful. It reformulates queries, plans, writes and explains, but every decision that affects trust (relevance, citation validity, grounding status, numeric correctness) is made by deterministic code. The same design produced three behaviours users rarely get from AI tools: the system says "I don't know", it reports its own degradation, and it shows how it reached its answer.

7.2 Limitations

- The calibration fixture is small (70 pairs, 7 chunks, one domain). The threshold is specific to the deployment and must be recalibrated when the corpus changes.
- There is no evaluation on a standard QA or attribution benchmark and no head-to-head comparison against a baseline RAG system.
- There has been no formal human evaluation of answer quality. Grounding status measures attribution, not correctness or usefulness.
- The relevance gate is a filter, not a classifier: the class distributions overlap, and 2 of 11 relevant pairs are rejected.
- Retrieval agents run sequentially, and latency and throughput have not been benchmarked.
- Analytics number checking verifies that a stated number *appears* in the results, not that it is attributed to the right quantity.

7.3 Ethical Considerations

Local-first processing keeps user documents on the user's infrastructure. Only synthesis prompts, which contain the selected evidence, are sent to cloud providers. The system never executes generated code and never presents a hypothesis as a finding. These choices limit the risk of fabricated results spreading into academic or business decisions.

Chapter 8 — Conclusion and Future Work

8.1 Conclusion

AIKDAP shows that an AI platform can move from *answering* to *completing*. It combines project-scoped memory, a planner-driven multi-agent architecture on LangGraph and a backend-enforced grounding chain. With these, it delivers research answers, analytics, reports, synopses and build plans whose claims are traceable, whose numbers are verified and whose execution is reconstructable. The calibrated relevance gate, citation validation and number grounding together turn "non-hallucination" from a hope expressed in a prompt into a property the system can show.

8.2 Future Work

1. Scale up calibration to multiple domains and thousands of labelled pairs, with automatic recalibration when the corpus changes.
2. Run benchmark evaluations on standard attribution and QA datasets against baseline RAG systems, plus human evaluation.
3. Fan out the retrieval agents in parallel, using the existing reducers, and benchmark latency.
4. Run experiments in a sandbox, behind an explicit review step, so experiment plans can be executed.
5. Bind numbers to semantic quantities in analytics validation.
6. Add more external sources (Semantic Scholar, arXiv) for paper suggestion.

References

1. A. Vaswani et al., "Attention Is All You Need," *NeurIPS*, 2017.
2. Z. Ji et al., "Survey of Hallucination in Natural Language Generation," *ACM Computing Surveys*, 2023.
3. P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *NeurIPS*, 2020.
4. V. Karpukhin et al., "Dense Passage Retrieval for Open-Domain Question Answering," *EMNLP*, 2020.
5. R. Nogueira and K. Cho, "Passage Re-ranking with BERT," arXiv:1901.04085, 2019.
6. J. Chen et al., "BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation," arXiv:2402.03216, 2024.
7. T. Gao et al., "Enabling Large Language Models to Generate Text with Citations," *EMNLP*, 2023.
8. S. Yao et al., "ReAct: Synergizing Reasoning and Acting in Language Models," *ICLR*, 2023.
9. Q. Wu et al., "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation," arXiv:2308.08155, 2023.
10. LangChain, Inc., "LangGraph Documentation," 2024–2026.
11. A. Adadi and M. Berrada, "Peeking Inside the Black-Box: A Survey on Explainable Artificial Intelligence (XAI)," *IEEE Access*, 2018.
12. J. Priem, H. Piwowar and R. Orr, "OpenAlex: A Fully-Open Index of Scholarly Works, Authors, Venues, Institutions, and Concepts," arXiv:2205.01833, 2022.
13. R. Smith, "An Overview of the Tesseract OCR Engine," *ICDAR*, 2007.